

AQUARIUM: Low-Latency Asynchronous BFT DAG-Based Consensus

Abstract—We present **AQUARIUM**, the first asynchronous BFT consensus protocol that achieves sub-second latency in a WAN setting while processing over 100,000 transactions per second. **AQUARIUM** achieves such high performance by leveraging an uncertified structured Directed Acyclic Graph (DAG) to forgo explicit certification. This reduces the number of messages required to commit and the CPU overhead for certificate verification, significantly. **AQUARIUM** introduces a novel commit rule that enables committing multiple blocks in each asynchronous DAG round. **AQUARIUM** can be parametrized either with a 5 message commit delay, maximizing the commit probability under a continuously active asynchronous adversary, or with a 4 message commit delay, reducing latency under a more moderate and realistic randomized adversary. We demonstrate safety and liveness of **AQUARIUM** in a Byzantine context for all of these parametrizations. Finally, we evaluate **AQUARIUM** in a geo-replicated setting and compare its performance to state-of-the-art asynchronous consensus protocols, showcasing **AQUARIUM**'s significantly lower latency.

Index Terms—Asynchronous Consensus, BFT, Blockchain

I. INTRODUCTION

Applications that require Byzantine Fault Tolerant (BFT) consensus [8], such as blockchains [3], [8], [10], [11], [32], [42], often rely on protocols designed for the partially synchronous network model which aims to approximate mostly benign network conditions. However, protocols designed for partial synchrony lose liveness under asynchronous conditions, which can arise from poor connectivity, an active network adversary, or denial-of-service (DoS) attacks [29]. Asynchronous consensus protocols [14], [16], [39] address this issue by providing as much liveness as the network connectivity allows. To achieve this, these protocols sacrifice performance during periods of network synchrony, resulting in significantly higher latency compared to their partially synchronous counterparts. While state-of-the-art partially synchronous protocols can process over 100,000 transactions per second with sub-second WAN latency [5], [6], current asynchronous protocols achieve similar throughput with latencies on the order of seconds [21]. This substantial latency drawback has made asynchronous consensus protocols less attractive for practical deployments.

Dual-mode protocols [28], [44] attempt to provide the best of both worlds by operating partially-synchronous consensus by default and reverting to a less performant asynchronous sub-protocol when network conditions become adverse. However, dual-mode protocols introduce complexity and are prone to errors, as they must maintain two separate protocol stacks and implement mechanisms to detect

changing network conditions and switch between the two consensus modes. Additionally, they remain vulnerable to targeted attacks that can cause the protocol to switch constantly between the two modes [44]. Due to these drawbacks, no dual-mode protocol has yet been deployed in a production environment to the best of our knowledge.

We therefore ask if it is possible to design a protocol that can simultaneously: (i) provide liveness under asynchronous network conditions, (ii) achieve performance comparable to state-of-the-art partially-synchronous consensus protocols, and (iii) maintain a simple design that allow for effective security analysis, implementation, and maintenance?

In this paper, we introduce **AQUARIUM**, a novel low-latency and high-throughput asynchronous consensus protocol that achieves these goals. **AQUARIUM** accomplishes this through a combination of the following techniques. (1) While state-of-the-art asynchronous protocols, such as Tusk [21], operate over a certified Directed Acyclic Graph (DAG) and attempt to commit one leader block every 9 message delays, **AQUARIUM** utilizes an *uncertified* DAG as its core data structure. This approach eliminates the overhead associated with the reliable broadcast [14] of DAG vertices and allows **AQUARIUM** to commit most blocks with only five message delays, aligning with the theoretical results of Cordial Miners [31]. (2) **AQUARIUM** introduces a novel commit rule that enables the commitment of multiple leader blocks in each DAG round while ensuring safety and liveness in the presence of an asynchronous adversary. (3) **AQUARIUM** also explores more practical network assumptions and can be parameterized to further enhance average-case performance while maintaining liveness against an asynchronous adversary.

We implement **AQUARIUM** in Rust and show that it can process an impressive 350,000 transactions per second in geo-distributed environments with 50 nodes, all while keeping latency below 2 seconds. Additionally, **AQUARIUM** can process 100,000 transactions per second with latency below 1 second. This achievement sets a new record in the realm of asynchronous consensus protocols and was previously only attainable by partially synchronous protocols [5], [6], [43]. We further show that **AQUARIUM** maintains the same throughput while reducing latency over recent state-of-the-art protocols, Tusk [21] and Cordial Miners [31] - achieving latency reductions of over 70% and 30%, respectively.

Contributions. We make the following contributions:

- We introduce **AQUARIUM**, the first asynchronous consensus protocol capable of committing with sub-second latency

while maintaining high throughput. Notably, AQUARIUM is the first DAG-based asynchronous consensus protocol capable of committing multiple leader blocks in each round.

- We provide detailed algorithms and formal security proofs for AQUARIUM, demonstrating its safety and liveness under an asynchronous network model.
- We conduct a latency analysis of AQUARIUM, evaluating its commit probability under various network conditions.
- We present an implementation and evaluation of AQUARIUM, comparing it to other state-of-the-art protocols and demonstrating that AQUARIUM achieves the lowest commit latency among available asynchronous protocols.

II. SYSTEM OVERVIEW

A. Threat model, goals, and assumptions

We consider a message-passing system with $n = 3f + 1$ validators. An adversary can adaptively corrupt up to f validators, referred to as *Byzantine*, who may deviate arbitrarily from the protocol. The remaining honest validators, follow the protocol. The adversary is computationally bounded, ensuring that standard cryptographic properties such as the security of hash functions and digital signatures hold. Under these assumptions, AQUARIUM is *safe* as no two correct validators commit different sequences of transactions. The communication network is asynchronous and messages can be delayed arbitrarily, but messages among honest validators are eventually delivered. Given these conditions AQUARIUM is *live*, meaning honest validators eventually commit transactions. We provide proofs in the long version of the paper [4].

In addition to the asynchronous model, we analyze AQUARIUM under the *random network model* [21], a variant of the asynchronous network model. While the asynchronous model makes the worst-case assumption that the adversary has perpetually full control over the message schedule (*i.e.*, the order in which messages are received by honest validators), the random network model assumes that the message schedule is random (see Section II-C). We analyze AQUARIUM with parameters optimized for the random network model, representing an average-case evaluation. Our empirical results show that this parameterization generally outperforms a version of AQUARIUM configured for maximum resilience against an asynchronous adversary, all while maintaining safety and liveness guarantees.

AQUARIUM solves Byzantine Atomic Broadcast (BAB) [17], enabling validators to reach consensus on a sequence of messages necessary for State Machine Replication (SMR). According to the FLP impossibility result [37], BAB cannot be solved deterministically in an asynchronous setting. To address this, we employ a global perfect coin to introduce randomization, similar to previous work [12], [15], [30], [34]. This coin can be constructed using an adaptively secure threshold signature scheme [7], [13], with the distributed key setup performed under asynchronous conditions [1], [2], [23].

Each validator v_k broadcasts messages by invoking $\text{a_bcast}_k(m, q)$, where m is the message and $q \in \mathbb{N}$ is a sequence number. Every validator v_i has an output

$\text{a_deliver}_i(m, q, v_k)$, where m is the message, q is the sequence number, and v_k is the identity of the validator that initiated the corresponding $\text{a_bcast}_k(m, q)$. AQUARIUM implements a BAB protocol and guarantees the following [30]:

- **Validity:** If an honest participant v_k calls $\text{a_bcast}_k(m, q)$, then every honest participant v_i eventually outputs $\text{a_deliver}_i(m, q, v_k)$, with probability 1.
- **Agreement:** If an honest participant v_i outputs $\text{a_deliver}_i(m, q, v_k)$, then every honest participant v_j eventually outputs $\text{a_deliver}_j(m, q, v_k)$ with probability 1.
- **Integrity:** For each sequence number $q \in \mathbb{N}$ and participant v_k , an honest participant v_i outputs $\text{a_deliver}_i(m, q, v_k)$ at most once, regardless of m .
- **Total Order:** If an honest participant v_i outputs $\text{a_deliver}_i(m, q, v_k)$ and $\text{a_deliver}_i(m', q', v'_k)$ where $q < q'$, all honest participants output $\text{a_deliver}_j(m, q, v_k)$ before $\text{a_deliver}_j(m', q', v'_k)$.

B. Intuition behind the AQUARIUM design

AQUARIUM builds upon DAG-based consensus protocols that achieve high throughput by processing $O(n)$ blocks per round. While maintaining these throughput advantages, AQUARIUM focuses on reducing latency in asynchronous state machine replication. It introduces novel techniques to decrease the number of message delays required for block commitment and explores more practical network assumptions to further improve average-case performance.

State-of-the-art asynchronous protocols, such as Tusk [21], operate over a certified DAG and commits one leader block every three certified rounds, requiring three message delays to certify each round. This results in at least nine message delays per leader block. To reduce latency, AQUARIUM operates over an uncertified DAG by forgoing the reliable broadcast [14] of DAG vertices, committing most blocks with only five message delays which matches the theoretical results of Cordial Miners [31]. This approach significantly reduces both bandwidth and computation cost.

This, however, creates the first challenge (**Challenge 1**): handling equivocations practically. Unlike certified DAG protocols [21], [27], [30], [44], [48], AQUARIUM cannot rely on certificates to prevent equivocations, necessitating the design of a novel commit rule to tolerate them. Cordial Miners [31] also faces this challenge, but they address it by eventually excluding Byzantine validators that provably equivocate, which can take a long time in asynchrony.

While having five rounds between leaders provides a good probability of committing in asynchronous conditions, it also results in relatively high latency. Although it can be shown that we can implement a commit rule that operates in just three message delays [6], this approach would not work under asynchrony as it would introduce significant latency variance and in the worst case lose liveness. Instead, we focus on addressing (**Challenge 2**): developing a commit rule that effectively reduces average-case latency without sacrificing worst-case liveness. We find that it is possible to reduce the number of rounds to four, achieving a balance

between average-case latency in random network conditions and worst-case latency in the classic asynchronous model.

Even with this enhancement, committing only once every four message delays still results in significant latency variance for transactions that are not part of a committed leader block. A primary goal for AQUARIUM is to commit multiple blocks in each round, which would ensure that the system’s tail latency aligns more closely with the four-message delay. To achieve this, we need to address (**Challenge 3**): commit every block without relying on a sufficient round difference between leader blocks. If AQUARIUM were to adopt a traditional recursive commit rule [21], [31], which mandates that each leader block always references all previous leader blocks in their causal history, it would at best be able to commit once every four rounds. However, AQUARIUM recognizes that this causal reference is only necessary when there is no sufficient evidence to directly commit a block, which is not the typical case (Section VI). This insight indicates that the recursive commit rule used in prior research is overly conservative in its approach to skipping blocks, leading to unnecessary delays, particularly during benign node crashes, which are immediately identifiable. To resolve this issue, we propose a new commit rule capable to promptly determine for each block whether it can be committed or discarded.

Section III-B presents the AQUARIUM commit rule that addresses these challenges. Hence, AQUARIUM is the first BFT consensus protocol capable of committing multiple blocks per round in the average case, while ensuring both safety and liveness in the asynchronous and random network models.

C. Structure of the AQUARIUM DAG

We present the structure of the AQUARIUM DAG, building an uncertified DAG that offers similar guarantees to a certified DAG, as shown in related work [6], [21], [31].

The AQUARIUM protocol operates in a sequence of logical *rounds*. In each round, every honest validator proposes a unique signed *block*, while Byzantine validators may attempt to equivocate by sending multiple blocks or none at all. During a round, validators receive transactions from users and blocks from other validators, which they refer into their proposed blocks. A block includes hash references to blocks from prior rounds, starting with their most recent block, and adds *fresh transactions* not yet included in preceding blocks. Once a block references at least $2f + 1$ blocks from the previous round, the validator signs it and broadcasts it. Clients send transactions to a validator, who adds them to their blocks. If a transaction does not finalize quickly enough, the client sends it to a different validator.

Block creation and validation. A block must include at least the following elements: (1) the author A of the block and their signature on the block contents; (2) a round number R ; (3) a list of transactions; (4) at least $2f + 1$ distinct hashes of valid blocks from the previous round $R - 1$, along with potentially others from prior rounds; and (5) a share of a global perfect coin. As already mentioned the coin can be reconstructed from any $2f + 1$ shares.

A block is *valid* if: (1) the signature is valid and the author A is part of the validator set; (2) all hashes point to distinct valid blocks from previous rounds, and the sequence of past blocks includes $2f + 1$ blocks from the previous round $R - 1$; and (3) the share of the global perfect coin is valid¹. Honest validators only include valid blocks into their DAG and discard invalid ones. Furthermore, honest validators only include hashes of blocks once they have downloaded their entire causal history, ensuring that they have successfully validated the block’s causal history.

Rounds and waves. Figure 1 (left) illustrates an example of a AQUARIUM DAG with four validators, (v_0, v_1, v_2, v_3) when parametrized to commit in 5 rounds. For the practically efficient 4-round AQUARIUM, the second Boost round is omitted.

In its 5-rounds configuration, AQUARIUM defines a *wave* of 5 rounds for every block. The first round (Propose) includes the blocks that the wave attempts to commit (P_0, P_1, P_2, P_3) and the equivocating block P'_1 . The second and third rounds (Boost) act as a buffer, helping to propagate these blocks to as many validators as possible. In the fourth round (Vote), every block serves as a *vote* for the first block of the Propose that it encounters when performing a depth- first search following the block hash references. In the example shown in this figure, blocks V_0, V_1 , and V_2 are votes for P_0, P_1, P_2 (but not for P'_1 and P_3), while block V_3 is a vote for P_0, P_1, P_2 , and P_3 (but not for P'_1). The fifth round (Certify) reveals which blocks from the Propose round have been implicitly certified. A block from the Propose round is considered *certified* or *has a certificate* if a block from the Certify round contains in its causal history at least $2f + 1$ blocks from the Vote round that are a vote for the block. In this example, blocks C_0, C_1, C_2 , and C_3 serve as certificates for P_0, P_1 , and P_2 . This round also opens the global perfect coin, which the decision rule (Section III-B) uses to circumvent the FLP result and to commit blocks under asynchrony by electing some of the Propose blocks as *leaders*. Similar to related work [30], [21] this strategy selects leaders “after the fact” to deter a network adversary from strategically delaying leader blocks so that they are not referenced by blocks of the Vote round.

As illustrated in Figure 1 (right), AQUARIUM initiates a new wave every round. The rounds of each wave follow a consistent pattern: Propose round: R , Boost round: $R + 1$, Boost round: $R + 2$, Vote round: $R + 3$, and Certify round: $R + 4$. This pattern repeats continuously, with each new round starting a fresh wave.

Random network model. We analyze AQUARIUM in the standard asynchronous network model, as well as in the random network model [21]. In the asynchronous model, the adversary chooses which blocks are received by each honest validator at each round. In contrast, the random network model assumes that at each round $R + 1$, an honest validator receives and references valid round- R blocks from

¹Each individual share of the coin can be independently verified if the coin is implemented through a threshold signature.

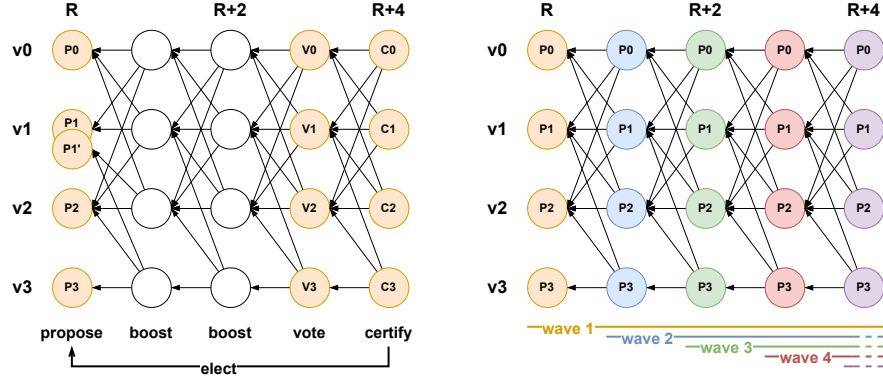


Fig. 1: The structure of the AQUARIUM DAG. Left: The structure of a wave, consisting of 5 rounds (Propose, Boost, Boost, Vote, Certify). Right: Waves patterns in the AQUARIUM protocol (each round starts a new overlapping wave).

a *uniformly random* subset of $2f + 1$ validators. The long version of the paper [4] provides further details and analyses the commit probability of AQUARIUM in both models.

III. THE AQUARIUM PROTOCOL

In this section, we present AQUARIUM configured with a wavelength of 5 rounds. A configuration of AQUARIUM with a wavelength of 4 rounds operates similarly, but omits one Boost round, and addresses **challenge 2** of Section II as we empirically show in Section VI.

A. Proposers and anchors

AQUARIUM leverages a perfect global coin to define several *leader slots* per round. A leader slot is a tuple (validator, round) and can be either empty or contain the validator's proposal for the respective round. If the validator is Byzantine, the slot may also contain more than one (equivocating) block. In line with related work [6], the slot can assume one of three states: commit, skip, or undecided. All slots are initially set to undecided and the goal of the protocol is to classify them as commit or skip. The number of leader slots instantiated per round and the number of boost rounds can be configured (Section VI explores different configurations).

B. The AQUARIUM decision rule

We present the decision rule of AQUARIUM leveraging an example protocol run. Figure 2 illustrates an example of a local view of a AQUARIUM validator, in a system with four validators, (v_0, v_1, v_2, v_3) and parameterized with two leader slots per round. In this example, we refer to blocks using the notation $B_{(v_i, R)}$, where v_i is the issuing validator and R is the block's round.

All proposer slots are initially in the undecided state. The validator holds the portion of the DAG depicted in Figure 2 and attempts to classify as many blocks in the leader slots as possible as either commit or skip.

Step 1: Determine the leader slots. The validator begins by reconstructing the global perfect coin to determine the leader slots for each round. As shown in Figure 1 (Left), the coin shares embedded in round $R + 4^2$ (the Certify round)

²Or $R + 3$ when AQUARIUM is configured with a wave length of 4 rounds.

deterministically establish the leader slots for round R .

In this example, the validator reconstructs the coin from any set of $2f + 1$ blocks from round $R + 4$ of a wave, then uses it as a seed to deterministically select two leader slots for round R : L_{1a} and L_{1b} , as illustrated in Figure 2. The coin also imposes an order between these two slots: by convention, L_{1a} is the *first* leader slot and L_{1b} is the *second* leader slot of round R . The validator repeats this process for every subsequent wave, determining leader slots L_{2a} and L_{2b} from the coin shares in round $R + 5$, L_{3a} and L_{3b} from those in round $R + 6$, and so on. The validator then sorts these leader slots in descending order: $[L_{6b}, L_{6a}, L_{5b}, L_{5a}, L_{4b}, L_{4a}, L_{3b}, L_{3a}, L_{2b}, L_{2a}, L_{1b}, L_{1a}]$.

This mechanism of determining multiple, potentially empty, leader slots from a global perfect coin is the first step towards addressing **challenge 3** (Section II). Even if validators have different views of the DAG, they will still deterministically decide the same leader slots, in the same order, for a given round—regardless of whether they have a block for that slot in memory. This enables AQUARIUM to achieve low latency by electing more than one leader per round and using these slots to order the causally history.

Step 2: Direct decision rule. The validator attempts to classify each slot, even those without a block as either commit or skip. To do so, the validator processes each slot individually, starting with the highest (L_{6b}), applying the AQUARIUM *direct decision rule*. The validator classifies a block B in a slot as skip if it observes $2f + 1$ blocks from the subsequent Vote round that do not encounter B when performing a depth-first search following the blocks' references, and as commit if it observes $2f + 1$ *certificates* over it. As discussed in Section II, a certificate over a block B is a block from the Certify round that references at least $2f + 1$ blocks from the Vote round, each of which encounter B when performing a depth-first search starting at the voting block. Otherwise, the validator leaves the slot as undecided (for now).

In this example, the validator targets L_{6b} first. It observes that $B_{(v_0, R+9)}$, $B_{(v_1, R+9)}$, and $B_{(v_2, R+9)}$ are certificates for L_{6b} . Therefore, it classifies L_{6b} as commit. Section VI shows that this scenario is the most common (in the absence of an

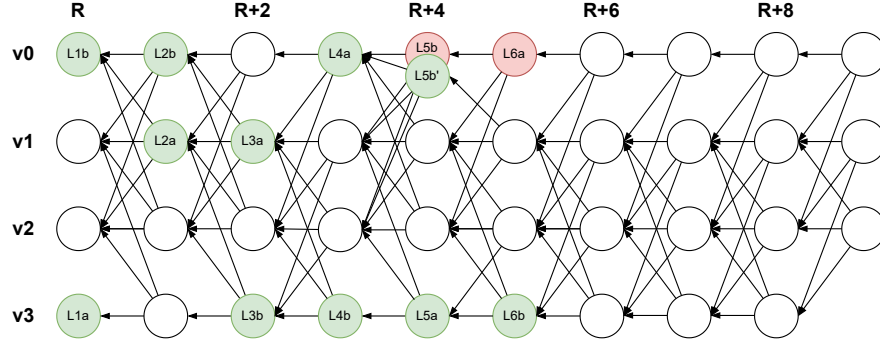


Fig. 2: Example execution with 4 validators, wave length of 5 rounds and 2 leader slots per round.

asynchronous adversary) and results in the lowest latency. The validator then targets L_{6a} and observes that $B_{(v_1, R+8)}$, $B_{(v_2, R+8)}$, and $B_{(v_3, R+8)}$ do not vote for it. Therefore, it classifies L_{6a} as skip. The presence of $2f + 1$ blocks from the Vote round that do not vote for a block ensures that it will never be certified, and will thus never be committed by other validators with a potentially different local view of the DAG. Section VI shows that this rule allows AQUARIUM to promptly skip (benign) crashed leaders to minimize their impact on the protocol's performance.

Malicious validators may attempt to equivocate by creating multiple blocks for the same slot, such as L_{5b} and L'_{5b} in this example. However, the direct decision rule ensures that at most one of these blocks will be classified as commit, while the others will be classified as skip. In this example, the block $B_{(v_0, R+7)}$ is a vote for L_{5b} (and not for L'_{5b}) as it is the first block of the slot encountered when performing a depth-first search starting at $B_{(v_0, R+7)}$ and recursively following all blocks in the sequence of block hashes. Conversely, $B_{(v_1, R+7)}$, $B_{(v_2, R+7)}$, and $B_{(v_3, R+7)}$ are votes for $L_{5b'}$.

This strategy addresses **challenge 1**. Even though Byzantine validators might equivocate by creating multiple blocks per slot, the causal references defined by the DAG allow the validator to interpret blocks from the Certify round as certificates for blocks from the Propose round. Coupled with the rule that honest validators author at most one block per round, this ensures that at most one block per slot receives a certificate, while all possible other equivocating blocks are skipped. In essence, AQUARIUM embeds the execution of a Byzantine consistent broadcast [14] into the DAG.

Step 3: Indirect decision rule. In the (rare) case where the direct decision rule cannot classify a slot, the validator uses the AQUARIUM *indirect decision rule*. This rule looks at future slots to decide about the current one. First, it finds an *anchor*. This is the first block of the next wave (that is, the earliest slot with a round number $R' > R + 4$) that is either still classified as undecided or already classified as commit. If the anchor is undecided, the validator marks the current slot as undecided. If the anchor is commit, the validator checks if it references at least one certificate over the current slot. If it does, the validator marks the current slot as commit. If it does not, the validator marks the current slot as skip.

In this example, the validator fails to classify L_{1a} using the

direct decision rule as there is only one certificate for L_{1a} and thus searches for its anchor. Since L_{6a} has been classified as skip, it cannot serve as an anchor; therefore, L_{6b} becomes the anchor for L_{1a} . Given that block $B_{(v_3, R+4)}$, which serves as a certificate for L_{1a} , is referenced in L_{6b} 's causal history, the validator classifies L_{1a} as commit.

This rule is the last step to solving **challenge 3**. It allows the validator to indirectly decide on a block by leveraging the earliest anchors rather than waiting for the next leader slot which may come much later. This enables AQUARIUM to eliminate the need for non-leader blocks between leader slots, achieving low latency by electing leader slots every round.

Step 4: Leader slots sequence. After processing all slots, the validator derives an ordered sequence of the blocks contained in the leader slots. It then iterates over this sequence, committing all slots marked as commit and skipping all slots marked as skip. This process continues until the validator encounters the first undecided slot. As shown in the long version of the paper [4], this commit sequence is safe, and eventually, all slots will be classified as either commit or skip.

In the example shown in Figure 2, the leader sequence output by the validator is $[L_{1a}, L_{1b}, L_{2a}, L_{2b}, L_{3a}, L_{3b}, L_{4a}, L_{4b}, L_{5a}, L'_{5b}, L_{6b}]$.

Step 5: Commit sequence. Following the approach introduced by DagRider [30], the validator linearizes the blocks within the sub-DAG defined by each leader block by performing a depth-first search. If a block has already been linearized by a previous leader slot, it is not re-linearized. The validator processes leader slots sequentially, ensuring that all blocks are included in the final commit sequence in the correct order, according to their causal dependencies.

In this example, L_{1a} and L_{1b} do not define any sub-DAG (the example begins at round R) and are thus directly added to the commit sequence. Next, L_{2a} defines the sub-DAG $\{L_{1b}, B_{(v_1, R)}, B_{(v_2, R)}, L_{2a}\}$, which is linearized as $[B_{(v_1, R)}, B_{(v_2, R)}, L_{2a}]$ since L_{1b} is already part of the commit sequence. The validator continues this process for each leader, linearizing the sub-DAGs defined by L_{3b} , then L_{3a} and so forth. The final commit sequence is $[L_{1a}, L_{1b}, B_{(v_1, R)}, B_{(v_2, R)}, L_{2a}, L_{2b}, B_{(v_2, R+1)}, L_{3a}, B_{(v_3, R+1)}, L_{3b}, B_{(v_0, R+2)}, B_{(v_2, R+2)}, L_{4a}, L_{4b}, B_{(v_1, R+3)}, B_{(v_2, R+3)}, L_{5a}, L'_{5b}, B_{(v_1, R+4)}, B_{(v_2, R+4)}, L_{6b}]$.

IV. SECURITY ANALYSES

The long version of the paper [4] contains a complete proof of correctness. Here, we state the main results and provide intuitive key ideas used in the proofs.

Theorem IV.1 (AQUARIUM Safety). *AQUARIUM satisfies the total order and integrity properties of Byzantine Atomic Broadcast.*

Theorem IV.2 (AQUARIUM Liveness). *AQUARIUM satisfies the validity and agreement properties of Byzantine Atomic Broadcast.*

Our proofs rely on the following key ideas:

- K1** Any two sets of $2f + 1$ validators must intersect in at least one honest validator, therefore: (i) once there is a certificate c for a block b , all future blocks must have a path to c , (ii) at most a single block per round and validator can be certified, (iii) if some honest validator commits a block, no other honest validator will skip that block, (iv) if some honest validator skips a block, no other honest validator will commit that block.
- K2** The common coin ensures that honest validators agree on the sequence of leader slots.
- K3** **K1** and **K2** together imply that honest validators agree on the order and contents of all decided slots (committed or skipped). This is the total order property.
- K4** The integrity property is ensured by construction: honest validators do not deliver more than one message per slot.
- K5** Since the network is reliable, any block b broadcast by an honest validator will eventually be received by all honest validators and included in their DAGs. Therefore, if v is an honest validator, v must eventually produce a block b_v that references b , and thus, all of v 's blocks after b_v will have a path to b .
- K6** Due to the fact that each valid block references at least $2f + 1$ blocks from the previous round, we can show through a pigeonhole argument that there must exist at each round r a *common core*, i.e., a block b such that all blocks at round $r + 2$ have a path to b .
- K7** In the $w = 5$ version of AQUARIUM, **K6** implies that in every wave, at least $2f + 1$ blocks from the Propose round have at least $2f + 1$ certificates each. To see this, note that the first Boost round of each wave must have a common core, by **K6**.
- K8** In the $w = 4$ version of AQUARIUM, we analyze the system under both the asynchronous model and the random network model. In the asynchronous model, **K6** implies that in every wave, at least one Propose round block has $2f + 1$ certificates in the Certify round. In the random network model, since the adversary no longer controls the message schedule, we can show that, with high probability, *all* Propose round blocks have at least $2f + 1$ certificates in the Certify round.
- K9** **K7** and **K8** imply that, in both the $w = 5$ and $w = 4$ version of AQUARIUM, there is a non-zero probability

that at least one leader slot is directly committed in each wave. To see this, consider that there is a non-zero probability that the common coin selects one or more of the Propose round blocks that have $2f + 1$ certificates each.

- K10** **K9** implies that each slot is eventually decided (i.e., committed or skipped) at each honest validator, with probability 1. To see this, assume that some slot s is never decided. Then it must be that s is not directly committed or skipped, and the same is holds true for the anchor of s , and for the anchor of the anchor of s , and so on. Since, by **K9**, there is a non-zero probability of committing a leader block in each wave, the probability of an infinite sequence of rounds without any decided leader slots is 0.
- K11** **K5**, **K9** and **K10** together imply the validity property: any block b broadcast by an honest validator is eventually included in the DAG of every honest validator by some time t . Eventually, with probability 1, some block is committed after t ; this block must have a path to b , so b is delivered by every honest validator.
- K12** **K1** and **K10** together imply the agreement property: a block b is delivered by an honest validator v when (1) v commits the first leader block l (potentially b itself) that has a path to b , and (2) all slots up to l have been decided. **K1** ensures that, if some honest validators commits l , then all honest parties commit l , and **K10** ensures that eventually, with probability 1, condition (2) is satisfied at all honest validators.

V. IMPLEMENTATION

We implemented a networked, multi-core AQUARIUM validator in Rust by forking the Mysticeti codebase [33], consisting of about 14,000 LOC. Our implementation utilizes tokio [47] for asynchronous networking and employs raw TCP sockets for communication. We rely on ed25519-consensus [24] for asymmetric cryptography and blake2 [40] for cryptographic hashing. To ensure data persistence and crash recovery, we implemented a Write-Ahead Log (WAL) tailored to the unique requirements of our consensus protocol. Furthermore, we implemented Cordial Miners [31], a state-of-the-art DAG-based asynchronous consensus protocol, using the same system components. This enabled us to to perform a comparative evaluation with AQUARIUM, see Section VI. Since the Cordial Miners paper lacks both implementation and evaluation, we believe our implementation and evaluation are additional contributions of our work. We are open-sourcing both our implementations of AQUARIUM and Cordial Miners, along with our orchestration tools, to ensure reproducibility of our results³.

VI. EVALUATION

We evaluate the throughput and latency of AQUARIUM through experiments conducted on Amazon Web Services (AWS), demonstrating its performance improvements over

³Code available but link omitted for blind review.

the state-of-the-art. We evaluate AQUARIUM with different parametrizations, with a wave length of 4 and 5 and with different numbers of leaders per round.

We compare AQUARIUM with Tusk [21], as an example of certified DAG-based consensus protocol, and Cordial Miners [31], as an example of an uncertified DAG-based protocol. We choose these protocols because, to the best of our knowledge, Tusk has shown the highest throughput among all published and implemented asynchronous BFT protocols when evaluated in a geo-distributed environment. Cordial Miners, while lacking an implementation and evaluation, theoretically proves excellent latency bounds and is the protocol most similar to AQUARIUM. We also considered a performance comparison with other asynchronous consensus protocols, including Pace [49], Fin [25], ParBFT [19], and SQ [46], but decided against them. The reasons for this decision is that their implementations are either closed-source, only capable of handling a limited number of block proposals (leading to crashes under sustained load), or unable to operate in a WAN environment (resulting in deadlocks after a few seconds).

Our evaluation demonstrates the following claims:

- C1 AQUARIUM has similar throughput and lower latency than the baseline state-of-the-art protocols when operating in synchronous network conditions.
- C2 AQUARIUM scales well by maintaining high throughput and low latency as the number of validators increases.
- C3 AQUARIUM has a similar throughput to, and lower latency than, Cordial Miners, when operating in the presence of (benign) crash faults.
- C4 AQUARIUM latency decreases when increasing the number of leader slots per round (up to 3 leaders per round).
- C5 AQUARIUM parametrized with a wave length of 4 rounds has lower latency in our geo-replicated network than when configured with a wave length of 5 rounds.

Note that evaluating the performance of BFT protocols in the presence of Byzantine faults is an open research question [9], and state-of-the-art evidence relies on formal proofs (presented in the long version of the paper [4]). While there is a need to robustly tolerate Byzantine faults, we note that they are rare in observed delegated proof-of-stake blockchains, as compared to crash faults which occur commonly [6].

A. Experimental Setup

We deploy AQUARIUM on AWS, using `m5d.8xlarge` instances across 5 different AWS regions: Ohio (us-east-2), Oregon (us-west-2), Cape Town (af-south-1), Hong Kong (ap-east-1), and Milan (eu-south-1). Validators are distributed across those regions as equally as possible. Each machine provides 10 Gbps of bandwidth, 32 virtual CPUs (16 physical cores) on a 3.1 GHz Intel Xeon Skylake 8175M, 128 GB memory, and runs Linux Ubuntu server 22.04. We select these machines because they provide decent performance and are in the price range of “commodity servers”.

In the following, *latency* refers to the time elapsed from the moment a client submits a transaction to when it is committed by the validators, and *throughput* refers to the

number of transactions committed per second. Each data point is the average latency of 3 runs and the error bars represent one standard deviation (error bars are sometimes too small to be visible on the graph). We instantiate several geo-distributed benchmark clients within each validator submitting transactions in an open loop model, at a fixed rate. We experimentally increase the load of transactions sent to the systems, and record the throughput and latency of commits. As a result, all plots illustrate the steady-state latency of all systems under low load, as well as the maximal throughput they can provide after which latency grows quickly. Transactions in the benchmarks are arbitrary and contain 512 bytes. Unless stated otherwise, we configure AQUARIUM with 2 leaders per round. In the following graphs, we refer to AQUARIUM with a wave length of 5 as AQUARIUM-5 and AQUARIUM with a wave length of 4 as AQUARIUM-4.

B. Benchmark under ideal conditions

We assess the performance of AQUARIUM under normal, failure-free conditions in a wide-area network (WAN). Figure 3 presents the performance results of AQUARIUM in a geo-replicated setting, comparing both a small committee of 10 validators and a large committee of 50 validators.

For a committee of 10 nodes, all three systems—Tusk, Cordial Miners, and AQUARIUM—reach a peak throughput of approximately 100k-130k transactions per second (tx/s). However, their latencies vary significantly. Tusk and Cordial Miners have average latency of 3.5s and 1.5s, respectively. In contrast, AQUARIUM with a wave length 5 has a latency of 1.1s, representing a reduction of 68% compared to Tusk and 27% compared to Cordial Miners. AQUARIUM with wave length 4 has a latency of 0.9s, representing a substantial reduction of 74% compared to Tusk and 40% compared to Cordial Miners. Tusk’s higher latency stems from its certified DAG architecture, requiring at least 9 network messages to commit a block. Cordial Miners bypasses DAG certification, but can only commit one leader every 5 rounds. By contrast, AQUARIUM operating with wave length 5 consistently commits multiple blocks. AQUARIUM operating with wave length 4 further reduces latency as it commits blocks after 4 message delays. These results validate claim C1.

For a large committee of 50 nodes, Figure 3 shows that the throughput of Cordial Miners and AQUARIUM exceeds 350,000 transactions per second (tx/s), while Tusk’s throughput remains around 125,000 tx/s. This perhaps surprising increase in throughput occurs because our AQUARIUM’s validator implementation is optimized for large networks and does not fully utilize all available resources (network, disk, CPU) when deployed with smaller committees. Consequently, adding more validators improves resource multiplexing, boosting AQUARIUM’s performance. Additionally, as the committee size grows, the number of blocks per round increases, thus a larger number of blocks are included in the causal history of elected leader blocks, without incurring additional network hops. Unlike Tusk, both Cordial Miners and AQUARIUM experience no significant CPU overhead as the committee

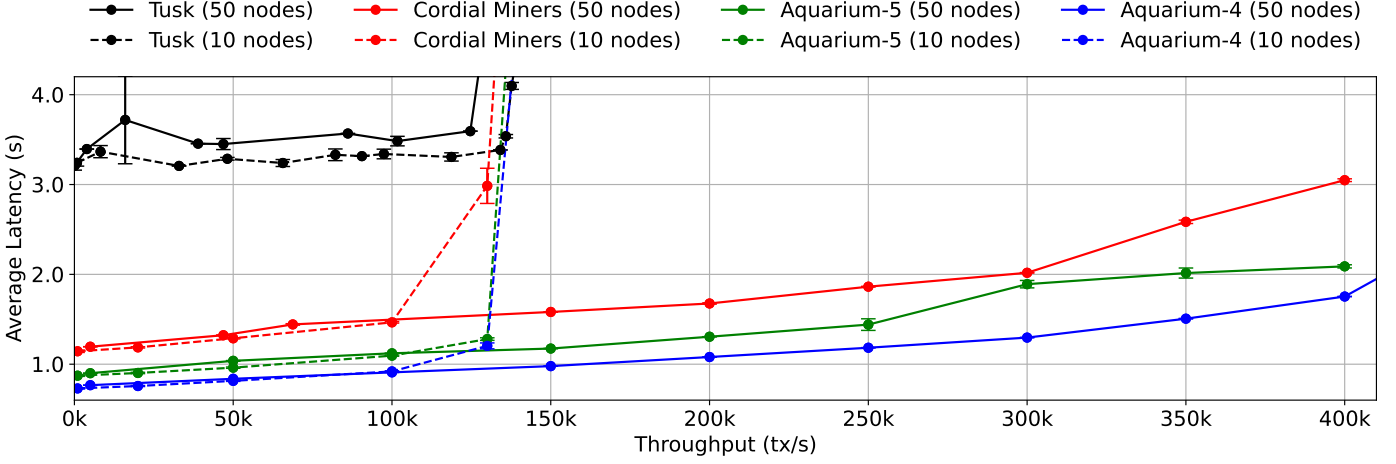


Fig. 3: Comparative throughput-latency performance of AQUARIUM, Tusk, and Cordial Miners. WAN measurements with 10 and 50 validators. No validator faults. 512B transaction size.

size increases, and bandwidth does not become a bottleneck at these throughput levels. However, we do not expect further throughput gains by increasing the committee size beyond 50 nodes (such experiments would be prohibitively expensive). As expected, Cordial Miners and AQUARIUM share nearly identical throughput since both rely on the same DAG implementation, and throughput is determined by the efficiency of the DAG propagation layer.

In terms of latency, Tusk and Cordial Miners achieve average latency of 3.5s and 2.6s, respectively. AQUARIUM parametrized with a wave length of 5 has a latency of 2s (at 350,000 tx/s), which is a 42% reduction compared to Tusk and a 23% reduction compared to Cordial Miners. AQUARIUM with a wave length 4 has a latency of 1.5s, which is a 57% reduction compared to Tusk and a 42% reduction compared to Cordial miners. These results validate our claim C2. Comparing the two versions of AQUARIUM in those two experiments also validates our claim C5.

C. Performance under faults

Figure 4 depicts the performance of all systems when a committee of 10 validators suffers 3 crash-faults (the maximum that can be tolerated for this committee size).

We observe that all three systems achieve a throughput of approximately 35,000-40,000 tx/s. Tusk and Cordial Miners record a latency of around 7s and 1.7s, respectively. AQUARIUM records a latency of 0.95s and 0.85s when running with a wave length 5 and 4, respectively. Despite the presence of faulty validators, the DAG continues to collect and disseminate transactions without significant impact. The reduction in throughput seen in Figure 4, compared to Figure 3, can be attributed to two primary factors: (1) the loss of capacity due to faulty validators, and (2) the higher frequency of missing elected leader blocks, which leads to increased commit delays. AQUARIUM maintains a latency advantage of approximately 50% over Cordial Miners, thanks to its direct skip rule (Section III), which allows AQUARIUM to bypass faulty leaders roughly 2 rounds earlier than Cordial Miners. Thus, our claim Item C3 holds.

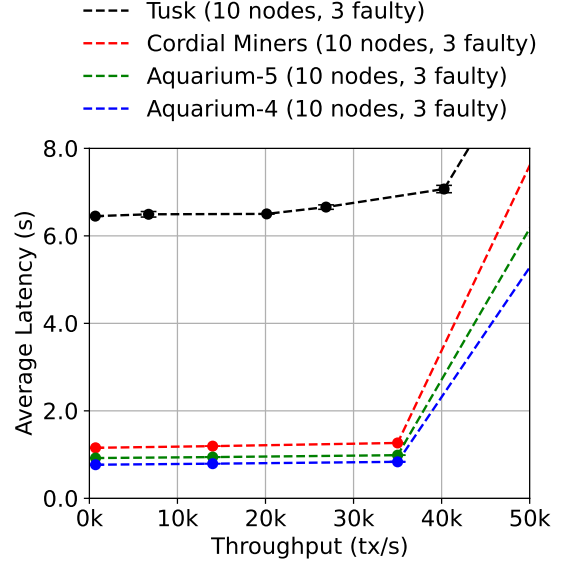


Fig. 4: Comparative throughput-latency of AQUARIUM, Tusk, and Cordial Miners. WAN measurements with 10 validators, three faults with 512B transactions size.

D. Impact of the number of leader slots per round

Finally, we assess the impact of multiple leaders on AQUARIUM's performance. We experiment with AQUARIUM parametrized with wave lengths of 4 and 5. Figure 5 illustrates how AQUARIUM configured with a wave length of 4 rounds performs with 1, 2, and 3 leaders under both normal conditions and scenarios involving 3 crash faults. The long version of the paper [4] shows the same experiment with wave length 5. We observe a notable reduction in average latency as the number of leaders increases. Specifically, when the number of leaders increases from 1 to 3, AQUARIUM's average latency decreases by approximately 40ms in the ideal scenario, and by approximately 100ms in the crash failure scenario. This improvement arises because having more leaders per round increases the number of blocks committed directly by leaders, rather than through the causal history of previous leader blocks. These findings validate our claim Item C4. Increasing the number of leaders beyond 3

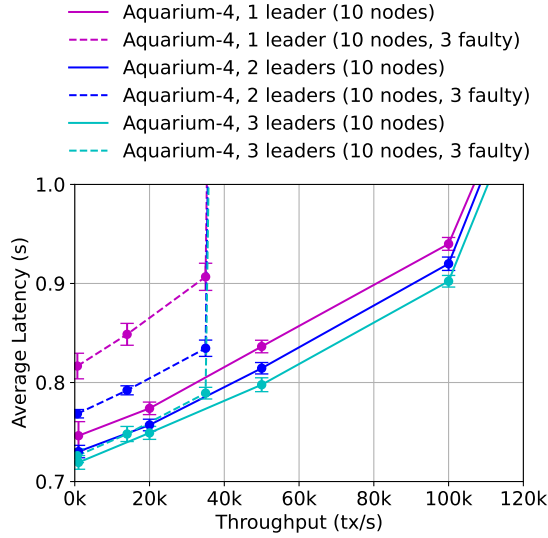


Fig. 5: Impact of the number of leaders per round in AQUARIUM. WAN measurements with 10 validators. Zero and three faults. 512B transaction size.

did not further decrease latency. This is due to the lower likelihood of directly committing, which may cause head-of-line blocking and delays the commitment of future leaders.

VII. RELATED WORK

Uncertified DAG-based consensus protocols. The system most similar to AQUARIUM is Cordial Miners [31]. Like AQUARIUM, Cordial Miners operates over an uncertified DAG, where each vertex represents a block that is disseminated with best-effort to all peers [26]. The primary distinction between the two lies in their commit rules. Cordial Miners can commit at most one leader block every five rounds, which leads to significantly higher latency for transactions not included in that leader block. In contrast, AQUARIUM’s commit rule allows for a configurable number of blocks to be committed in each round, increasing the number of blocks committed per round and reducing the latency for most transactions. AQUARIUM commits more blocks directly through leaders, rather than relying on the causal history of previous leader blocks. Additionally, Cordial Miners does not provide an implementation or evaluation.

Mysticeti [6] is a recent protocol that, like AQUARIUM, operates over an uncertified DAG but in a partially synchronous setting. Mysticeti takes advantage of synchronous periods in the network to commit blocks in three rounds, and like AQUARIUM, it can commit blocks every round. However, unlike AQUARIUM, Mysticeti completely loses liveness when the network is not synchronous. To maintain liveness in asynchronous conditions, AQUARIUM interprets the DAG differently from Mysticeti. Specifically, AQUARIUM incorporates a global perfect coin into the protocol and modifies the role of several DAG rounds to ensure that an asynchronous adversary cannot indefinitely manipulate message schedules to prevent block certificates from forming—an issue that can easily arise in Mysticeti [29].

Certified DAG-based consensus protocols. DAG-

TABLE I: Comparison of asynchronous DAG-based consensus protocols

	LB Lat.	NLB Lat.	Complexity	Leaders
DAG-Rider [30]	12	24	$O(n^3)$	One
Tusk [21]	9	15	$O(n^3)$	One
Cordial Miners [31]	5	10	$O(n^2)$	One
GradedDAG [20]	5 (4) [†]	10 (9) [†]	$O(n^3)$	One
LightDAG1 [18]	6 (5) [†]	10 (9) [†]	$O(n^3)$	One
LightDAG2 [18]	4	8	$O(n^3)$	One
AQUARIUM (this work)	5 (4) [‡]	5 (4) [‡]	$O(n^2)$	Many

LB Lat. and **NLB Lat.** stand for *leader block latency* and *non-leader block latency*, respectively. Latency refers to the number of message delays to commit a block in the best case. Complexity refers to the number of messages sent per DAG round. [†]GradedDAG and LightDAG can decide early, after the first phase of a two-phase Byzantine consistent broadcast instance; the value in parentheses is the latency when deciding early. [‡]The value in parentheses shows AQUARIUM’s latency in the random network model.

Rider [30], Tusk [21], and Dumbo-NG [27] are popular asynchronous certified DAG-based consensus protocols that use reliable or consistent broadcast to explicitly certify every DAG vertex [38]. This approach introduces 3 message delays per DAG round but simplifies the commit rule by ensuring that equivocating DAG vertices never occur. However, this method results in significantly higher latency compared to AQUARIUM. For instance, DAG-Rider [30], GradedDAG [20], and LightDAG [18] require at least 12 messages to commit a block, while Tusk and Dumbo-NG require 9 message delays. By contrast, AQUARIUM can commit in just 4 or 5 message delays when respectively configured with a wave length of 4 and 5. Also, certified DAGs have higher bandwidth and CPU requirements, as validators must disseminate, receive, and verify the cryptographic certificates generated by consistent broadcast. As shown in Section VI, these factors lead to up to 70% higher latency in comparison to AQUARIUM.

Sailfish [41], BBKA-Chain [35], Fino [36], Shoal [43], and Shoal++ [5] build on the partially synchronous version of Bullshark [45] through various improvements, including the ability to commit more blocks per round and a relaxation of DAG certification requirements. However, these protocols are limited to partially synchronous environments and, unlike AQUARIUM, they lose liveness in asynchronous conditions.

Linear-chain protocols. Linear-chain asynchronous protocols such as Das *et al.* [22], Pace [49], FIN [25], and SQ [46] do not leverage an underlying DAG structure. They instead rely on explicit Byzantine consistent broadcast [14] and a common coin to elect a leader, whereas AQUARIUM incorporates these components implicitly within the DAG. This leader drives the protocol by constructing a linear chain. Consequently, these protocols do not achieve the same level of throughput and robustness as DAG-based systems [21]. Their contributions instead lie primarily in their theoretical foundations. For example, Das *et al.* introduces a protocol that operates without a trusted setup or the need for public-key cryptography; FIN presents the first constant-time asynchronous consensus (ACS) protocol with $O(n^3)$ messages in both information-theoretic and signature-free settings; and SQ reduces this message complexity to $O(n^2)$.

VIII. CONCLUSION

We introduce **AQUARIUM**, an asynchronous consensus protocol achieving a new performance milestone: an impressive 350,000 transactions per second in geo-distributed environments with 50 nodes all while keeping latency below 2 seconds, or 100,000 transactions per second with sub-second latency—an achievement that sets a new record in the realm of asynchronous consensus protocols and that was only thought possible for partially-synchronous protocols. The exceptional performance is made possible through a novel commit rule applied over an uncertified DAG that enables commits of multiple leaders every round. This allows **AQUARIUM** to inherit the robustness and throughput inherent in DAG-based protocols, while establishing a new standard for the latency of asynchronous consensus protocols.

REFERENCES

- [1] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, and Gilad Stern. Bingo: Adaptivity and Asynchrony in Verifiable Secret Sharing and Distributed Key Generation. In *CRYPTO*, 2023.
- [2] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. Reaching Consensus for Asynchronous Distributed Key Generation. *Distributed Computing*, 2023.
- [3] Mustafa Al-Bassam, Alberto Sonnino, Shehar Bano, Dave Hryczyn, and George Danezis. Chainspace: A sharded smart contracts platform. *arXiv preprint arXiv:1708.03778*, 2017.
- [4] Anonymous. Aquarium: Low latency DAG based BFT consensus. <https://drive.google.com/file/d/1S96wqPW-zSJ3K6nchhQupSj7FWUDkRvL/view>, 2024.
- [5] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. Shoal+: High throughput DAG BFT can be fast! *CoRR*, abs/2405.20488, 2024.
- [6] Kushal Babel, Andrey Chursin, George Danezis, Lefteris Kokoris-Kogias, and Alberto Sonnino. Mysticeti: Low-latency DAG consensus with fast commit path. *CoRR*, abs/2310.14821, 2023.
- [7] Renas Bacho and Julian Loss. On the Adaptive Security of the Threshold BLS Signature Scheme. In *ACM CCS*, 2022.
- [8] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. Consensus in the age of blockchains. *CoRR*, abs/1711.03936, 2017.
- [9] Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. Twins: Bft systems made robust. In *ACM PODC*, 2021.
- [10] Mathieu Baudet, George Danezis, and Alberto Sonnino. Fastpay: High-performance byzantine fault tolerant settlement. In *ACM AFT*, 2020.
- [11] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, and et al. Sui lutris: A blockchain combining broadcast and consensus. *CCS*, 2023.
- [12] Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. Asynchronous byzantine agreement with subquadratic communication. In *ACM TCC*, 2020.
- [13] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In *ASIACRYPT*, 2001.
- [14] Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. Introduction to reliable and secure distributed programming, 2011.
- [15] Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography. In *ACM PODC*, 2000.
- [16] Junchao Chen, Alberto Sonnino, Lefteris Kokoris-Kogias, and Mohammad Sadoghi. Thunderbolt: Causal concurrent consensus and execution. *CoRR*, abs/2407.09409, 2024.
- [17] Flaviu Cristian, Houtan Aghili, Ray Strong, and Danny Dolev. Atomic Broadcast: From Simple Message Diffusion to Byzantine Agreement. *Information and Computation, Volume 118, Issue 1*, 1995.
- [18] Xiaohai Dai, Guanxiong Wang, Jiang Xiao, Zhengxuan Guo, Rui Hao, Xia Xie, and Hai Jin. LightDAG: A Low-latency DAG-based BFT Consensus through Lightweight Broadcast. In *IEEE IPDPS*, 2024.
- [19] Xiaohai Dai, Bolin Zhang, Hai Jin, and Ling Ren. ParBFT: Faster Asynchronous BFT Consensus with a Parallel Optimistic Path. In *ACM CCS*, 2023.
- [20] Xiaohai Dai, Zhaonan Zhang, Jiang Xiao, Jingtao Yue, Xia Xie, and Hai Jin. GradedDAG: An Asynchronous DAG-based BFT Consensus with Lower Latency. In *SRDS*, 2023.
- [21] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *ACM EuroSys*, 2022.
- [22] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. Asynchronous consensus without trusted setup or public-key cryptography. In *ACM CCS*, 2024.
- [23] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. Practical Asynchronous High-threshold Distributed Key Generation and Distributed Polynomial Sampling. In *USENIX Security*, 2023.
- [24] Henry de Valence. Ed25519 for consensus-critical contexts. <https://crates.io/crates/ed25519-consensus>, 2024.
- [25] Sisi Duan, Xin Wang, and Haibin Zhang. FIN: Practical Signature-Free Asynchronous Common Subset in Constant Time. In *ACM CCS*, 2023.
- [26] Bryan Ford. Threshold logical clocks for asynchronous distributed coordination and consensus. *CoRR*, abs/1907.07010, 2019.
- [27] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency. In *ACM CCS*, 2022.
- [28] Rati Gelashvili, Lefteris Kokoris-Kogias, Alberto Sonnino, Alexander Spiegelman, and Zhuolun Xiang. Jolteon and Ditto: Network-Adaptive Efficient Consensus with Asynchronous Fallback. In *FC*, 2022.
- [29] Giacomo Giuliani, Alberto Sonnino, Marc Frei, Fabio Streun, Lefteris Kokoris-Kogias, and Adrian Perrig. An Empirical Study of Consensus Protocols’ DoS Resilience. In *ACM ASIACCS*, 2024.
- [30] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. All You Need is DAG. In *ACM PODC*, 2021.
- [31] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *DISC*, 2023.
- [32] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding. In *IEEE S&P*, 2018.
- [33] Mysten Labs. Mysticeti: Low-latency dag consensus with fast commit path. <https://github.com/asonnino/mysticeti>, 2024.
- [34] Julian Loss and Tal Moran. Combining asynchronous and synchronous byzantine agreement: The best of both worlds. *IACR ePrint*, 2018.
- [35] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. BBCHAIN: One-Message, Low Latency BFT Consensus on a DAG. In *FC*, 2024.
- [36] Dahlia Malkhi and Pawel Szalachowski. Maximal extractable value (mev) protection on a dag. In *Tokenomics*, 2022.
- [37] Michael S. Paterson Michael J. Fischer, Nancy A. Lynch. Impossibility of distributed consensus with one faulty process. *Journal of ACM*, 1985.
- [38] Mayank Raiikwar, Nikita Polyanski, and Sebastian Müller. SoK: DAG-based Consensus Protocols. In *IEEE ICBC*, 2024.
- [39] Zhijie Ren, Kelong Cong, Johan Pouwelse, and Zekeriya Erkin. Implicit Consensus: Blockchain with Unbounded Throughput, 2017.
- [40] RustCrypto. Hashes. <https://github.com/RustCrypto/ hashes>, 2024.
- [41] Nibesh Shrestha, Aniket Kate, and Kartik Nayak. Sailfish: Towards improving latency of dag-based BFT. *IACR Cryptol. ePrint Arch.*, 2024.
- [42] Alberto Sonnino, Shehar Bano, Mustafa Al-Bassam, and George Danezis. Replay attacks and defenses against cross-shard consensus in sharded distributed ledgers. In *IEEE EuroS&P*, 2020.
- [43] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. Shoal: Improving dag-bft latency and robustness. In *FC*, 2024.
- [44] Alexander Spiegelman, Neil Gridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: DAG BFT Protocols Made Practical. In *ACM CCS*, 2022.
- [45] Alexander Spiegelman, Neil Gridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: the partially synchronous version. *arXiv preprint arXiv:2209.05633*, 2022.
- [46] Xiao Sui and Sisi Duan. Signature-free atomic broadcast with optimal $O(n^2)$ messages and $O(1)$ expected time. *IACR ePrint*, 2023.
- [47] The Tokio Team. Tokio. <https://tokio.rs>, 2024.
- [48] Lei Yang, Seo Jin Park, Mohammad Alizadeh, Sreeram Kannan, and David Tse. DispersedLedger: High-throughput byzantine consensus on variable bandwidth networks. In *USENIX NSDI*, 2022.
- [49] Haibin Zhang and Sisi Duan. Pace: Fully parallelizable bft from reposable byzantine agreement. In *ACM CCS*, 2022.

Algorithm 1 AQUARIUM Main Function

```

1: waveLength ▷ Set to at least 4 (see Section III)
2: leadersPerRound ▷ See Section VI for details

// Called by the application layer to extend the commit sequence (idempotent).
3: procedure EXTENDCOMMITSEQUENCE( $r_{committed}, r_{highest}$ )
4:    $L \leftarrow \text{TRYDECIDE}(r_{committed}, r_{highest})$  ▷ See Line 11 below
5:    $L_{commit} \leftarrow []$  ▷ Hold decided leader sequence
6:   for  $status \in L$  do
7:     if  $status = \perp$  then break ▷ Stop at the first undecided leader
8:     if  $status = \text{commit}(b_{leader})$  then
9:        $L_{commit} \leftarrow L_{commit} || b_{leader}$ 
10:  return LINEARIZESUBDAGS( $L_{commit}$ ) ▷ See Line Line 20 of Algorithm 2

// Try to decide proposals, recursively, starting from the latest proposal.
11: procedure TRYDECIDE( $r_{committed}, r_{highest}$ )
12:    $L \leftarrow []$  ▷ Hold decision of each leader
13:   for  $r \in [r_{highest} \text{ down to } r_{committed} + 1]$  do
14:     for  $l \in [\text{leadersPerRound} - 1 \text{ down to } 0]$  do ▷ All possible leaders
15:        $i \leftarrow r \% \text{waveLength}$ 
16:        $D \leftarrow \text{Decider}(\text{waveLength}, i, l)$  ▷ Algorithm 3
17:        $w \leftarrow D.\text{WAVELENGTH}(r)$ 
18:       if  $D.\text{PROPOSEROUND}(w) \neq r$  then continue ▷ Skip if not a leader
19:        $status \leftarrow D.\text{TRYDIRECTDECIDE}(w)$  ▷ Direct decision rule
20:       if  $status = \perp$  then
21:          $status \leftarrow D.\text{TRYINDIRECTDECIDE}(w)$  ▷ Indirect decision rule
22:        $L \leftarrow status || L$ 
23:  return  $L$  ▷ May still contain undecided leaders

```

Algorithm 2 DAG Helper Functions

```

1: procedure ISVOTE( $b_{vote}, b_{leader}$ )
2:   function VOTEDBLOCK( $b, id, r$ )
3:     if  $r \geq b.\text{round}$  then return  $\perp$ 
4:     for  $b' \in b.\text{parents}$  do
5:       if  $(b'.\text{author}, b'.\text{round}) = (id, r)$  then return  $b'$ 
6:        $res \leftarrow \text{VOTEDBLOCK}(b', id, r)$ 
7:       if  $res \neq \perp$  then return  $res$ 
8:     return  $\perp$ 
9:    $(id, r) \leftarrow (b_{leader}.\text{author}, b_{leader}.\text{round})$ 
10:  return  $\text{VOTEDBLOCK}(b_{vote}, id, r) = b_{leader}$ 

11: procedure ISCERT( $b_{cert}, b_{leader}$ )
12:    $res \leftarrow |\{b \in b_{cert}.\text{parents} : \text{ISVOTE}(b, b_{leader})\}|$ 
13:  return  $res \geq 2f + 1$ 

14: procedure ISLINK( $b_{old}, b_{new}$ )
15:  return exists a sequence of  $k \in \mathbb{N}$  blocks  $b_1, \dots, b_k$  s.t.  $b_1 = b_{old}, b_k = b_{new}$  and  $\forall j \in [2, k] : b_j \in \bigcup_{r \geq 1} \text{DAG}[r, *] \wedge b_{j-1} \in b_j.\text{parents}$ 

16: procedure ISCERTIFIEDLINK( $b_{anchor}, b_{leader}$ )
17:    $w \leftarrow \text{WAVELENGTH}(b_{leader}.\text{round})$ 
18:    $B \leftarrow \text{GETDECISIONBLOCKS}(w)$ 
19:  return  $\exists b \in B$  s.t.  $\text{ISCERT}(b, b_{leader}) \wedge \text{ISLINK}(b, b_{anchor})$ 

20: procedure LINEARIZESUBDAGS( $L$ )
21:    $O \leftarrow []$  ▷ Hold output sequence
22:   for  $b_{leader} \in L$  do
23:      $B \leftarrow \{b \in \bigcup_{r \geq 1} \text{DAG}[r, *] \text{ s.t. } \text{ISLINK}(b, b_{leader}) \wedge b \notin O \wedge b \text{ not already output}\}$ 
24:     for  $b \in B$  in any deterministic order do
25:        $O \leftarrow O || b$ 
26:  return  $O$ 

```

APPENDIX

This section presents the algorithms used in AQUARIUM in pseudocode format. If a high-level understanding of AQUARIUM is sufficient, then this section can be skipped. In particular, Algorithm 1 specifies the AQUARIUM main algorithms, Algorithm 3 the AQUARIUM decider instance, and algorithm 2 contains various DAG helper functions. The long version of the paper [4] briefly describes these algorithms.

Algorithm 3 AQUARIUM Decider Instance

```

1: waveLength ▷ Set to at least 4 (see Section III)
2: waveOffset ▷ Offset creating overlapping waves (Section II)
3: leaderOffset ▷ Each decider operates on a unique leader slot

4: procedure WAVELENGTH( $r$ )
5:   return  $(r - \text{waveOffset}) / \text{waveLength}$ 

6: procedure PROPOSEROUND( $w$ )
7:   return  $w \cdot \text{waveLength} + \text{waveOffset}$  ▷ See Figure 1

8: procedure CERTIFYROUND( $w$ )
9:   return  $w \cdot \text{waveLength} + \text{waveLength} - 1 + \text{waveOffset}$  ▷ See Figure 1

10: procedure VOTEROUND( $w$ )
11:  return  $\text{Self.CERTIFYROUND}(w) - 1$  ▷ See Figure 1

12: procedure LEADERBLOCK( $w$ )
13:    $r_{propose}, r_{certify} \leftarrow \text{Self.PROPOSEROUND}(w), \text{Self.CERTIFYROUND}(w)$ 
14:    $c \leftarrow \text{COMBINECOINSHARES}(\{b.\text{share} \text{ s.t. } b \in \text{DAG}[r_{certify}, *]\})$ 
15:    $l \leftarrow c + \text{leaderOffset}$  ▷ Modulo committee size
16:  return  $\text{DAG}[r_{propose}, l]$  ▷ May return more than one block in case of equivocations

17: procedure SKIPPEDLEADER( $w, b_{leader}$ )
18:    $r_{vote} \leftarrow \text{Self.VOTEROUND}(w)$ 
19:  return  $|\{\neg \text{ISVOTE}(b, b_{leader}) \text{ s.t. } b \in \text{DAG}[r_{vote}, *]\}| \geq 2f + 1$ 

20: procedure SUPPORTEDLEADER( $w, b_{leader}$ )
21:    $r_{certify} \leftarrow \text{Self.CERTIFYROUND}(w)$ 
22:  return  $|\{\text{ISCERT}(b, b_{leader}) \text{ s.t. } b \in \text{DAG}[r_{certify}, *]\}| \geq 2f + 1$ 

23: procedure TRYDIRECTDECIDE( $w$ )
24:   for  $b_{leader} \in \text{Self.LEADERBLOCK}(w)$  do ▷ Loop over equivocations
25:     if  $\text{Self.SKIPPEDLEADER}(w, b_{leader})$  then return  $\text{skip}(w)$ 
26:     if  $\text{Self.SUPPORTEDLEADER}(w, b_{leader})$  then return  $\text{commit}(b_{leader})$ 
27:  return  $\perp$ 

28: procedure TRYINDIRECTDECIDE( $w, S$ )
29:    $s_{anchor} \leftarrow \text{find first } s \in S \text{ s.t. } r_{certify} < s.\text{round} \wedge s \neq \text{skip}(w)$ 
30:   if  $s_{anchor} = \text{commit}(b_{anchor})$  then
31:     if  $\exists b_{leader} \in \text{Self.LEADERBLOCK}(w) \text{ s.t. } \text{ISCERTIFIEDLINK}(b_{anchor}, b_{leader})$ 
32:       return  $\text{commit}(b_{leader})$ 
33:     else
34:       return  $\text{skip}(w)$ 
35:  return  $\perp$  ▷ The anchor is undecided or not found

```
